

“Design and Development of a Web-Based Aircraft Maintenance Tracker for Centralized Fleet Maintenance Management”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

CH.V.S.MANIKANTA VARMA RAJU :2520030239

B.VASUDEV :2520090136

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering

K L (Deemed to be) University

Declaration

The Project Report entitled “Design and Development of a Web-Based Aircraft Maintenance Tracker for Centralized Fleet Maintenance Management” is a record of bonafide work of CH.V.S.MANIKANTA VARMA RAJU – 2520030239, B.VASUDEV – 2520090136 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

CH.V.S.MANIKANTA VARMA RAJU – 2520030239
B.VASUDEV – 2520090136

Certificate

This is to certify that the mini project based report entitled “Design and Development of a Web-Based Aircraft Maintenance Tracker for Centralized Fleet Maintenance Management” is a bonafide work done and submitted by CH.V.S.MANIKANTA VARMA RAJU – 2520030239, B.VASUDEV – 2520090136 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2024-2025.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

Acknowledgement

The success of this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or another for the completion of this project.

Our greatest appreciation goes to our Course Coordinator and our guide, Dr Yerragudipadu Subbarayudu, Department of Computer Science, for his tremendous support, encouragement and guidance throughout this project.

We express our gratitude to Dr. P Venkateshwara Rao (CSE) / Dr. Kaja Shareef (CSIT), Head of the Department, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly, and finally we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

CH.V.S.MANIKANTA VARMA RAJU – 2520030239
B.VASUDEV – 2520090136

Abstract

Aircraft maintenance is one of the most safety-critical functions in the aviation industry, yet many maintenance teams still rely on paper logs, spreadsheets and scattered communication channels to track tasks, technicians and spare parts. This fragmented approach leads to missed deadlines, poor visibility into aircraft status, delayed parts procurement and difficulty proving regulatory compliance. To address this, our project focuses on the Design and Development of a Web-Based Aircraft Maintenance Tracker — a centralized, single-window platform that brings scheduled and unscheduled maintenance, technician assignment, parts inventory, status tracking and reporting into one streamlined system.

The application is built around core interconnected modules: an Admin Panel for user and role setup, a Task List module for creating and assigning scheduled (A/B/C/D checks, engine overhauls) and unscheduled (defects, pilot-reported issues) maintenance tasks, a Schedule Calendar for visual planning, an Alert/Reminder system for upcoming and overdue work, Technician Assignment with workload balancing, Parts Inventory tracking with low-stock alerts, real-time Status Updates, Photo Upload for proof of completed work, a searchable Maintenance History log, and an Export Report module that produces PDF/Excel records for audits.

The system was implemented as a fully responsive single-page web application using React.js and modern front-end tooling, with client-side state management used to simulate real-time data flow between modules. In summary, this project delivers a reliable, easy-to-use digital solution that improves task accountability, reduces missed maintenance, speeds up incident response, and supports the regulatory and safety obligations of an airline maintenance operation.

Index

S.No.	Chapter	Topics
		Acknowledgement / Abstract
1	Introduction	1.1 Background 1.2 Problem Statement 1.3 Scope 1.4 Objectives 1.5 Importance 1.6 Target Users
2	System Requirements	2.1 Hardware 2.2 Software 2.3 Development Tools & Frameworks
3	Technology Stack	3.1 Front-end 3.2 Back-end / Data Layer 3.3 Storage 3.4 Version Control 3.5 Libraries & Services
4	System Architecture	4.1 High-Level Architecture 4.2 Layer Description 4.3 Deployment Architecture
5	Design	5.1 Module Flow Diagram 5.2 Data Schema
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 Data/API Integration 6.4 Authentication & Authorization
7	Features	7.1 List of Main Features 7.2 How Each Feature Works
8	Testing	8.1 Functional Testing 8.2 Integration Testing 8.3 Tools Used 8.4 Test Cases & Results
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting
10	Challenges & Limitations	10.1 Issues Faced 10.2 Solutions Applied 10.3 Current Limitations
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations
12	Conclusion	12.1 Summary 12.2 What Was Achieved 12.3 Skills Learned
13	References	Books, tutorials, APIs, documentation sites used
14	Appendices	Screenshots, code snippets, setup instructions, user manual

Chapter 1: Introduction

1.1 Background of the Project

Airlines and maintenance, repair and overhaul (MRO) teams are legally required to keep every aircraft airworthy through a strict regime of scheduled inspections (A, B, C and D checks), component overhauls, and rapid response to defects reported during flight operations. Traditionally, much of this coordination has happened through paper work-cards, whiteboards, spreadsheets and phone calls between supervisors, technicians and the stores department. As fleets and maintenance volumes grow, this manual approach becomes error-prone and difficult to audit.

The Aircraft Maintenance Tracker was conceived as a centralized web application that digitizes this entire workflow — from task creation to technician assignment, parts consumption, photo-based proof of work, and final report generation — so that maintenance operations can be planned, tracked and audited from a single dashboard.

1.2 Problem Statement

Maintenance teams currently face several recurring issues: manual tracking leads to missed deadlines and overdue inspections; maintenance records are scattered across multiple documents and departments; there is no real-time visibility into the status of any given aircraft or task; coordinating and assigning technicians to the right job is done informally and inefficiently; and parts shortages are frequently discovered only at the point of need, causing avoidable delays. There is a clear need for a single, reliable system that solves all of these problems together.

1.3 Scope of the Project

The scope of this project covers the design and development of a browser-based maintenance management application that supports: creation and tracking of scheduled and unscheduled maintenance tasks; calendar-based scheduling; an automated alert/reminder system; technician assignment and workload tracking; parts inventory management with low-stock alerts; status updates with photo documentation; a searchable maintenance history log; and exportable PDF/Excel reports. The system is designed for use by maintenance administrators, supervisors and technicians within a single maintenance organization, and the current implementation focuses on the front-end application layer with client-side data handling, leaving large-scale multi-base backend integration as a future enhancement.

1.4 Objective(s)

1. To design a centralized platform that unifies all maintenance-related data in one place.
2. To enable creation, assignment and tracking of both scheduled and unscheduled maintenance tasks.
3. To provide real-time visibility into task status, technician workload and parts availability.
4. To reduce missed maintenance deadlines through an automated alert and reminder system.
5. To maintain a complete, auditable maintenance history with photographic proof of work performed.
6. To allow administrators to manage users, roles and permissions securely.

7. To generate exportable reports that support regulatory compliance and audits.

1.5 Importance of the Application

Aviation maintenance directly affects passenger and crew safety, so any improvement in traceability, accountability and timeliness has an outsized impact. By centralizing maintenance data, the application reduces the risk of overlooked inspections, shortens the time between a defect being reported and a technician being dispatched, and makes it far easier for an organization to demonstrate compliance with aviation authority record-keeping requirements during an audit.

1.6 Target Users / Audience

- Maintenance Administrators – configure users, roles and overall system settings.
- Maintenance Supervisors – create tasks, assign technicians, monitor progress and generate reports.
- Technicians – view assigned tasks, update status, upload proof photos and log parts used.
- Viewers / Management – read-only access to dashboards, history and reports for oversight and decision-making.

Chapter 2: System Requirements

2.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core 1.6 GHz or higher
RAM	4 GB minimum (8 GB recommended)
Storage	256 MB free space for the application and local data
Display	1280 x 720 resolution or higher
Network	Standard broadband / Wi-Fi connection for asset loading and updates

2.2 Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS, or Linux
Browser	Latest version of Chrome, Edge, Firefox or Safari
Runtime (development)	Node.js 18+ and npm
Code Editor	Visual Studio Code
Version Control	Git

2.3 Development Tools and Frameworks

- React.js – component-based front-end framework used to build the user interface.
- Vite – fast development server and build tool used to bundle the application.
- JavaScript (ES6+) / JSX – primary application logic and component markup.
- CSS3 – styling, layout and responsive design.
- Git & GitHub – source control and collaborative development.
- Visual Studio Code – primary integrated development environment.
- Browser DevTools – debugging, performance profiling and responsive testing.

Chapter 3: Technology Stack

3.1 Front-end

The user interface is built entirely with React.js, using functional components and React hooks (useState, useEffect, useContext) for state management. Vite is used as the build tool and development server for fast hot-module reloading. The interface is styled with custom CSS3 for a responsive, dashboard-style layout that adapts to desktop, tablet and mobile screens, and uses chart components to visualize task completion and inventory statistics.

3.2 Back-end / Application Logic

In the current version, application and business logic (task lifecycle rules, alert generation, technician workload calculation, low-stock detection) is implemented on the client side within React components and utility modules. This keeps the application lightweight and easy to demo, while the architecture is designed so that this logic layer can be moved to a dedicated Node.js/Express service in a future iteration without major redesign (see Chapter 11, Future Enhancements).

3.3 Data Storage

Application data — users, tasks, technicians, parts inventory and maintenance history — is persisted using the browser's local storage during this phase of development, allowing the application to retain state across sessions without requiring a server. The data model is structured so that it can be migrated to a relational database such as MySQL/PostgreSQL or a document store such as MongoDB when a server-side back-end is introduced.

3.4 Version Control

Git is used for version control, with the project repository hosted on GitHub to track changes, manage feature branches and coordinate work between the three team members.

3.5 Libraries, Tools and Services

- React.js – UI library / component framework
- Vite – build and bundling tool
- Charting library – for dashboard analytics (task completion rate, cost/time analysis)
- Icon library – for consistent iconography across modules
- PDF/Excel export utilities – for the Export Report module
- ESLint – code quality and linting during development

Chapter 4: System Architecture

4.1 High-Level Architecture Diagram

The diagram below shows the end-to-end functional flow of the Aircraft Maintenance Tracker, from initial admin setup through to final report export.



4.2 Description of Each Layer

Presentation Layer (Front-end)

Built with React.js, this layer renders the Admin Panel, Task List, Calendar, Alerts, Technician Assignment, Parts Inventory, Status Updates, Photo Upload, Maintenance History and Export Report screens. It handles all user interaction and client-side validation.

Application / Logic Layer

Implemented through React state management and utility functions, this layer enforces business rules — such as preventing a technician from being double-booked, triggering low-stock alerts, and computing task completion percentages.

Data Layer

Currently implemented using browser local storage, structured around entities such as Users, Tasks, Technicians, Parts and MaintenanceLogs. The data layer is abstracted behind a small set of access functions so that it can later be swapped for REST API calls to a relational database.

4.3 Deployment Architecture

The application is built as a static single-page application using Vite's production build (the dist/ bundle), which can be deployed to any static hosting provider or CDN (such as Netlify, Vercel, GitHub Pages, or an internal IIS/Apache server) without requiring a dedicated application server. A future server-backed deployment would add a Node.js/Express API layer and a

managed database, fronted by the same React build, with CI/CD handled through GitHub Actions.

Chapter 5: Design

5.2 Data Schema

The core data entities and their key attributes are summarized below. These mirror the structure used in local storage today and map directly onto future relational tables.

Entity	Key Fields
User	userId, name, email, role (Admin/Supervisor/Technician/Viewer), passwordHash, status
Task	taskId, aircraftId, type (Scheduled/Unscheduled), title, description, priority, status, dueDate, createdBy
Technician	technicianId, name, skillSet, certification, availability, assignedTasks, performanceHistory
Part	partId, name, partNumber, quantityInStock, reorderLevel, supplier, usageHistory
MaintenanceLog	logId, taskId, technicianId, statusChange, photoUrl, remarks, timestamp
Alert	alertId, relatedTaskId, type (Reminder/LowStock/Overdue), message, createdAt, isRead

Relationships: a Task is linked to one Aircraft and may be linked to one or more Technicians and Parts; a MaintenanceLog entry references a single Task and the Technician who performed the update; an Alert may reference a Task or a Part depending on its trigger condition.

Chapter 6: Implementation

6.1 Module-wise Implementation

- Admin Panel – allows creation of user accounts and assignment of roles (Admin, Supervisor, Technician, Viewer) before any other module can be used.
- Task List – supervisors create scheduled or unscheduled maintenance tasks with priority, description and due date.
- Schedule Calendar – a visual calendar view plots all open tasks by date to support planning and avoid scheduling conflicts.
- Alert System – background checks compare task due dates and part stock levels against thresholds and raise reminder notifications.
- Technician Assignment – supervisors allocate technicians to tasks based on skill set and current workload.
- Parts Inventory – tracks quantity on hand for each part, decrements stock as parts are consumed, and flags items below the reorder level.
- Status Update – technicians update task progress (Pending, In Progress, Completed, On Hold) in real time.
- Photo Upload – technicians attach photographic evidence of completed work directly to the task record.
- Maintenance History – a searchable, filterable chronological log of every task and status change for audit purposes.
- Export Report – generates downloadable PDF/Excel summaries of tasks, parts usage and technician performance.

6.2 Front-end Logic (UI Rendering, State Management)

Each module is implemented as an independent React component tree. Shared application state (current user, task list, technician list, inventory) is lifted to common parent components and passed down through props and context, so that an update in one module (for example, marking a task complete) is immediately reflected wherever that data is shown (the dashboard, the calendar and the maintenance history).

6.3 Data / API Integration

Data access is centralized through a small set of helper functions that read from and write to local storage, mimicking the request/response pattern of a REST API. This design choice means the helper layer can later be replaced with real HTTP calls to a Node.js/Express backend with minimal changes to the React components themselves.

6.4 Authentication & Authorization

Users sign in with an email and password validated against the stored user records. Once authenticated, the user's role determines which modules and actions are visible: Admins have full access; Supervisors can manage tasks, technicians and reports; Technicians can only view and update tasks assigned to them; Viewers have read-only access across the system.

Chapter 7: Features

7.1 List of Main Features

Feature	Summary
Task Management	Create, assign and track scheduled and unscheduled maintenance tasks.
Calendar Planning	Visual work calendar for organized scheduling and deadline management.
Alert System	Automated reminders for upcoming, due and overdue maintenance items.
Technician Assignment	Allocate skilled technicians to jobs with workload balancing.
Parts Inventory	Track spare-parts availability with auto-alerts for low stock.
Status Updates	Real-time task progress tracking with instant notifications.
Photo Documentation	Upload images as visual proof of completed maintenance work.
Maintenance History	Full, searchable chronological log of all maintenance activity.
Export Report	Generate downloadable PDF/Excel reports for audits and analysis.
Admin Panel & Roles	User management with Admin, Supervisor, Technician and Viewer access levels.

7.2 How Each Feature Works

Task Management

From the user perspective, a supervisor opens the Task List, clicks “New Task”, and fills in the aircraft, maintenance type (scheduled or unscheduled), priority and description. Technically, the new task object is appended to the tasks collection and immediately reflected in the calendar and dashboard views.

Calendar Planning

Users see all open tasks plotted against a monthly/weekly calendar. Internally, each task’s due date is used to position it on the calendar grid, allowing supervisors to visually spot clustering or conflicts.

Alert System

The system continuously evaluates due dates and stock levels; when a task is within its reminder window or a part falls below its reorder threshold, an alert is generated and surfaced in the notification panel.

Technician Assignment

Supervisors select a technician from a filtered list (filtered by skill and current workload) and assign them to a task; the technician’s task queue and the task’s status update simultaneously.

Parts Inventory

Each part record tracks current quantity; when parts are logged as used against a task, the quantity is decremented automatically, and the inventory screen highlights any item that has crossed its low-stock threshold.

Status Updates & Photo Documentation

Technicians change a task's status as work progresses and can attach photos at any stage; these photos and status changes are written to the maintenance log so there is a complete, timestamped record.

Maintenance History & Export Report

All historical log entries can be searched by aircraft, technician, date range or task type, and the filtered result set can be exported as a PDF or Excel file for compliance reporting or management review.

Chapter 8: Testing

8.1 Functional / Manual Testing

Each module was manually tested against its expected behavior — for example, verifying that a task created as “Unscheduled” appears immediately with the correct priority flag, and that completing a task updates its status across the calendar and history views simultaneously.

8.2 Integration Testing

Cross-module flows were tested end to end: creating a task → assigning a technician → logging parts usage → updating status to Completed → verifying the entry appears correctly in Maintenance History and in the exported report, confirming that state changes propagate correctly between components.

8.3 Tools Used for Testing

- Browser DevTools – console, network and component state inspection.
- React DevTools – component tree and state/props inspection.
- Manual cross-browser testing – Chrome, Edge and Firefox.
- Responsive design testing – device emulation for tablet and mobile breakpoints.

8.4 Test Cases and Results

Test Case	Expected Result	Status
Create a scheduled task	Task appears in Task List and Calendar with correct due date	Pass
Create an unscheduled task	Task is flagged high priority and triggers an immediate alert	Pass
Assign technician to task	Technician workload count increases; task shows assignee	Pass
Log part usage below reorder level	Low-stock alert is generated on Parts Inventory	Pass
Mark task as Completed	Status updates everywhere; entry added to Maintenance History	Pass
Upload photo to task	Photo attached and visible in task detail and history log	Pass
Export maintenance report	PDF/Excel file downloads with correct filtered data	Pass
Login with Technician role	Only assigned tasks are visible; admin screens hidden	Pass

Chapter 9: Deployment

9.1 Steps to Deploy

8. Install dependencies with `npm install`.
9. Run `npm run build` to generate the optimized production bundle in the `dist/` folder.
10. Upload the contents of `dist/` to a static hosting provider (e.g., Netlify, Vercel, GitHub Pages) or an internal web server.
11. Configure the hosting provider to serve `index.html` for all routes (single-page application fallback).
12. Verify the deployed build by checking that all modules load correctly and local storage persists data as expected.

9.2 Environment Configuration

The application currently requires no environment variables or external API keys, as it operates entirely on the client with local storage as its data layer. When a back-end service is introduced, environment configuration will include the API base URL, database connection string and authentication secrets, managed through a `.env` file excluded from version control.

9.3 Hosting of the Application

The production build is a static bundle (HTML, CSS and JavaScript assets) suitable for any CDN-backed static host. This keeps hosting costs minimal and deployment straightforward, while leaving room to add a back-end API service on a separate host once server-side persistence is implemented.

Chapter 10: Challenges & Limitations

10.1 Issues Faced During Development

- Designing a data model that could realistically represent scheduled and unscheduled maintenance without a server-side database.
- Keeping state synchronized across multiple modules (calendar, alerts, history) that all depend on the same task data.
- Building a calendar view that correctly handles tasks spanning multiple days and overlapping schedules.
- Ensuring the UI remained responsive and usable on smaller tablet screens used by technicians on the maintenance floor.

10.2 Solutions Applied

- Centralized shared state in common parent components and used context to avoid prop-drilling across modules.
- Adopted a consistent data-access helper layer so all modules read/write through the same functions, keeping data consistent.
- Used CSS Grid/Flexbox with responsive breakpoints to adapt layouts for tablet and mobile use.

10.3 Current Limitations / Known Issues

- Data is stored in browser local storage, so it is not currently shared across devices or users in real time.
- There is no server-side authentication; the current login is suitable for demonstration but not production-grade security.
- Photo uploads are stored client-side and are not yet optimized for very large image files.
- Reporting currently covers standard PDF/Excel exports; advanced analytics (e.g., predictive maintenance) is not yet implemented.

Chapter 11: Future Enhancements

11.1 Planned Features

- A dedicated Node.js/Express back-end with a relational database to enable true multi-user, multi-device real-time synchronization.
- Push and email/SMS notifications for critical alerts in addition to in-app notifications.
- Role-based audit logging for every change made within the system, viewable by Admins.
- A mobile companion app for technicians to update task status directly from the hangar floor.

11.2 Possible Integrations and Optimizations

- Integration with airline scheduling systems to auto-detect upcoming A/B/C/D check windows.
- Integration with supplier systems for automated parts reordering when stock crosses the reorder threshold.
- Predictive maintenance analytics using historical task and failure data.
- Single sign-on (SSO) integration for enterprise identity management.

Chapter 12: Conclusion

12.1 Summary of the Project

This project set out to design and build a centralized, web-based Aircraft Maintenance Tracker capable of replacing fragmented manual processes with a single, unified system for managing scheduled and unscheduled maintenance, technician assignment, parts inventory and reporting.

12.2 What Was Achieved

The team successfully delivered a fully functional, responsive React.js web application covering all ten core modules identified in the original design — Admin Panel, Task List, Schedule Calendar, Alert System, Technician Assignment, Parts Inventory, Status Updates, Photo Upload, Maintenance History and Export Report — along with role-based access for Admin, Supervisor, Technician and Viewer users.

12.3 Skills Learned During Development

- Practical experience building a multi-module single-page application with React.js and component-based architecture.
- State management patterns for keeping interdependent modules in sync.
- UI/UX design for dashboard-style, data-heavy enterprise applications.
- Project planning, version control and collaborative development practices using Git/GitHub.
- An appreciation for the real-world operational and safety requirements behind aviation maintenance software.

Chapter 13: References

- React.js Official Documentation – <https://react.dev>
- Vite Official Documentation – <https://vitejs.dev>
- MDN Web Docs – JavaScript, CSS and Web APIs reference – <https://developer.mozilla.org>
- Git & GitHub Documentation – <https://docs.github.com>
- Federal Aviation Administration (FAA) – Aircraft Maintenance Regulations, general reference for domain context.
- Project team study material and lecture notes, Department of Computer Science Engineering, K L University.

Chapter 14: Appendices

14.1 Screenshots of the Application

Representative screens include the Admin Panel, Task List, Schedule Calendar, Technician Assignment board, Parts Inventory dashboard, and Maintenance History log. (Screenshots to be inserted from the running application build located in the project's dist/ folder.)

14.2 Sample Code Snippet — Task Status Update Handler

```
function updateTaskStatus(taskId, newStatus) {  const tasks = getTasks();
const updated = tasks.map(t =>    t.id === taskId ? { ...t, status:
newStatus, updatedAt: Date.now() } : t    );  saveTasks(updated);
logMaintenanceEvent(taskId, newStatus); }
```

14.3 Installation / Setup Instructions

13. Clone the repository: `git clone <repository-url>`
14. Navigate into the project folder: `cd "FEED Project"`
15. Install dependencies: `npm install`
16. Start the development server: `npm run dev`
17. Open the application in a browser at the local address shown in the terminal (typically `http://localhost:5173`).
18. To create a production build, run `npm run build`, then preview it with `npm run preview`.

14.4 User Manual / Guide (Brief)

- Login using your assigned email and password; the Admin creates accounts via the Admin Panel.
- Supervisors: use Task List to create tasks, Calendar to plan, and Technician Assignment to allocate work.
- Technicians: open My Tasks, update status as work progresses, and attach photos on completion.
- Use Maintenance History to search past records and Export Report to download PDF/Excel summaries.